

Programa de Pós-Graduação em Computação Aplicada  
Universidade de Brasília

# **Geração Automatizada de Testes Baseada em Model Checking: Um Estudo Aplicado ao e-Título Versão para Qualificação**

Thiago Viana Fernandes  
thiagu.vf@gmail.com

24 de novembro de 2025

Introdução

Introdução

Justificativa e Objetivos

Estado da Arte

Fundamentação Teórica

Metodologia

Modelagem do Onboarding

Aplicação do SPIN e Propriedades

Geração de Casos de Teste

Resultados e Análises

Conclusão e Trabalhos Futuros

Referências

- ▶ Importância da qualidade de software em sistemas críticos.
- ▶ Limitações dos testes tradicionais para explorar todos os cenários possíveis.
- ▶ Motivação para o uso de Model Checking como técnica complementar.
- ▶ Contexto do *e-Título* como sistema crítico do TSE.

**Tabela:** Quantidade de downloads do aplicativo e-Título

| Ano  | Eleitoral | iOS         | Android     |
|------|-----------|-------------|-------------|
| 2019 | Não       | 328.397     | 14.292.413* |
| 2020 | Sim       | 4.240.474   | 17.505.366  |
| 2021 | Não       | 1.434.842   | 4.498.665   |
| 2022 | Sim       | 11.538.108  | 19.512.395  |
| 2023 | Não       | 3.429.769   | 4.203.042   |
| 2024 | Sim       | 13.611.919  | 15.924.420  |
| 2025 | Não       | 4.565.176** | 5.040.111** |

Tendo um total de 57.143.504 de eleitores com e-Título ativo em novembro de 2025.

- ▶ Dificuldade em identificar falhas emergentes de interações entre componentes.
- ▶ Ausência de integração consolidada entre testes automatizados e verificação formal.
- ▶ Pergunta de pesquisa: a integração de Model Checking com testes melhora a detecção de falhas?

- ▶ *e-Título* atende milhões de eleitores e exige alta confiabilidade.
- ▶ Fluxos complexos (síncronos e assíncronos) aumentam riscos não cobertos por testes convencionais.
- ▶ Model Checking complementa testes ao identificar interações e estados indesejados.

- ▶ **Objetivo Geral:** Guiar a definição de testes automatizados a partir de análises por Model Checking.
- ▶ **Objetivos Específicos:**
  - ▶ Investigar limitações dos testes automatizados atuais.
  - ▶ Explorar a viabilidade do Model Checking como complemento.
  - ▶ Definir um método semi-automatizado para geração de testes.
  - ▶ Aplicar e validar a metodologia no estudo de caso do *e-Título*.

- ▶ Revisão baseada em survey sobre testes de APIs REST e trabalhos posteriores.
- ▶ Crescimento recente do uso de IA (LLMs) para geração e oráculos de teste.
- ▶ Lacuna: poucas integrações entre técnicas formais (model checking) e frameworks de teste.



- ▶ **Cobertura:** esquema (OpenAPI), código, métricas especializadas.
- ▶ **Detecção de falhas:** erros 5xx, violações de esquema, violação de regras definidas.
- ▶ **Desempenho:** tempo de resposta e latência (menos explorado).

- ▶ Testes tendem a focar em cenários previsíveis e "happy path".
- ▶ Oráculos fracos e restrições ocultas reduzem efetividade.
- ▶ Pouca exploração de verificação formal aplicada a APIs REST em contexto industrial.

- ▶ Testes como atividades de Verificação, Validação e Teste (VV&T).
- ▶ Diferença entre verificação (conforme especificação) e validação (produto certo).
- ▶ Limitações de testes unitários e importância de testes de integração para sistemas distribuídos.

- ▶ Exploração exaustiva do espaço de estados de um modelo formal.
- ▶ Ferramentas: SPIN (Promela) e PRISM (modelos probabilísticos).
- ▶ Dependência da qualidade do modelo e da formulação de propriedades (LTL).

- ▶ Model Checking identifica cenários logicamente possíveis que violam propriedades.
- ▶ Transformação de violações em contratos (Design by Contract) e casos de teste.
- ▶ Geração de testes que validam se o sistema implementado realmente evita esses cenários.

- ▶ Abordagem iterativa (PDCA): modelagem, verificação, extração de caminhos, teste.
- ▶ Critérios de avaliação: cobertura, detecção de falhas e aplicabilidade prática.
- ▶ Ferramentas e recursos: SPIN, Promela, ferramentas de teste para APIs REST.

1. Construção do modelo Promela do fluxo alvo (onboarding).
2. Definição de propriedades LTL representando requisitos e invariantes.
3. Execução do SPIN e análise de contra-exemplos / violações.
4. Tradução de contra-exemplos em casos de teste automatizados (ex.: BDD).

- ▶ Etapas modeladas: preenchimento de dados biográficos, carrossel de perguntas, senha e verificação biométrica.
- ▶ Uso de processos concorrentes em Promela para representar interações reais.
- ▶ Geração de grafo de estados para análise de propriedades e métricas.



- ▶ Aplicação de métricas especializadas para avaliar representatividade do modelo.
- ▶ Mapeamento entre endpoints REST e transições do modelo (tabela de mapeamento).
- ▶ Importância do alinhamento entre implementação e modelo.

- ▶ Propriedades LTL formuladas para capturar comportamentos esperados (ex.: sequências inválidas).
- ▶ Verificação busca contra-exemplos, ciclos de aceitação e violações.
- ▶ Tradução das propriedades violadas em cláusulas de Design by Contract (DbC).

- ▶ Observação de acceptance cycles e análise das causas.
- ▶ Extração de caminhos de execução a partir de contra-exemplos.
- ▶ Conversão dos passos do contra-exemplo em passos de um caso de teste BDD.

- ▶ Cada violação no modelo descreve um cenário lógico possível.
- ▶ Cenários são convertidos em scripts de teste automáticos ou em BDD (Given-When-Then).
- ▶ Integração com frameworks de teste para executabilidade contra a API real.

- ▶ Contra-exemplo: usuário que falha no carrossel e consegue pular etapa X.
- ▶ Conversão: sequência de chamadas REST e dados de entrada para reproduzir o cenário.
- ▶ Resultado: teste detecta comportamento indevido no sistema real.

- ▶ Identificação de inconsistências no fluxo de onboarding do *e-Título*.
- ▶ Tradução sistemática de LTL violadas para contratos DbC e casos de teste.
- ▶ Demonstração de aplicabilidade da metodologia em um caso real.

- ▶ Dependência da qualidade e fidelidade do modelo Promela.
- ▶ Escalabilidade do Model Checking para modelos muito grandes.
- ▶ Necessidade de revisão humana nos artefatos gerados (ex.: scripts de teste).

- ▶ A integração entre Model Checking e testes automatizados aumenta a robustez da validação.
- ▶ Método aplicável ao *e-Título* com resultados promissores.
- ▶ Recomenda-se aumento da automação e ampliação do escopo (mais fluxos e serviços).



- ▶ Automação completa da transformação de contra-exemplos em testes executáveis.
- ▶ Explorar modelos probabilísticos (PRISM) para situações estocásticas.
- ▶ Integração com ferramentas de IA para auxiliar na geração de dados de teste e oráculos.

- ▶ Dissertação: Thiago Viana Fernandes (2025). “Geração Automatizada de Testes Baseada em Model Checking: Um Estudo Aplicado ao e-Título”. (arquivo fonte em: </mnt/data/56327998-111e-4b62-aa48-b2f9c8428b40.pdf>)
- ▶ Seleção de trabalhos citados na dissertação (survey sobre testes de APIs, SPIN, Promela, DbC).

Obrigado!